

Sub-queries

A parenthesized SELECT-FROM-WHERE statement is a **sub-query**. It can be used as a value in several places:

Where it goes	What it acts like
in FROM	a temporary table you must give it a tuple-variable (alias).
in WHERE, returning <i>one</i> tuple of <i>one</i> value	a single value, usable with =, <, >, etc.
in WHERE, returning a <i>column</i> of values	a set, usable with IN, ANY, ALL.

Mental model: run the inner query first, treat its result as a table or value, then run the outer query.

Bag vs. Set Semantics

A **set** has no duplicates. A **bag** (multi-set) allows them.

SELECT ... FROM ... WHERE	bag , duplicate rows are kept.
SELECT DISTINCT ...	duplicates removed, result is a set.
UNION, INTERSECT, EXCEPT	set by default duplicates removed.

Think of it like math: {1, 2, 2, 3} as a set is {1, 2, 3}; as a bag it stays {1, 2, 2, 3} and |bag| = 4.

Aggregation

An aggregation function collapses a *column* of values into *one* value. It is a **column procedure**, not a row procedure.

COUNT(*)	number of rows.
COUNT(expr)	number of non-NULL values.
COUNT(DISTINCT expr)	number of distinct non-NULL values.
AVG / SUM / MIN /	average / total / smallest / largest.
MAX(expr)	

Rules. Aggregates may appear in SELECT (and HAVING), **never** in WHERE. GROUP BY col splits rows into groups sharing the same col; the aggregate is then computed *once per group*.

IN, ANY, ALL

These compare *one value* against a *set* of values (often a sub-query result).

v IN (S)	true if v equals <i>some</i> element of S.
v op ANY (S)	true if v op x holds for <i>at least one</i> x in S.
v op ALL (S)	true if v op x holds for <i>every</i> x in S.

op is any comparison (=, <, >, <>, ...). Note v IN (S) is the same as v = ANY (S), and v <> ALL (S) is the same as v NOT IN (S).

Our Schema (used on Side B)

Underlines mark key attributes.

Students(sid, name, major, gpa) Courses(cid, title, dept, credits) Takes(sid, cid)

Students				Takes		Courses			
sid	name	major	gpa	sid	cid	cid	title	dept	credits
1	Anya	CS	3.8	1	CSC108	CSC108	Intro to CP	Comp Sci	5
2	Boris	CS	3.2	1	MAT136	CSC148	Data Structures	Comp Sci	5
3	Carmen	Math	3.9	2	CSC108	MAT136	Calculus I	Math	5
4	Diego	Math	3.5	3	CSC148	MAT223	Linear Algebra I	Math	4
5	Elif	Bio	3.4	3	MAT136	BIO120	Adaptation	Biology	3
				4	CSC108				

Use Side A's schema and sample data. Write actual SQL, not pseudocode. Boxes marked (*discuss*) are debriefed with the class after individual work.

Box 1 Sub-queries, Step by Step (8 min, discuss)

Goal: the titles of every course taken by a Math major.

(a) First write the *inner* query: the **sid** of every Math major. List the rows its result contains.

(b) Its result is a temporary table. Place it in the **FROM** clause of an outer query, give it the tuple-variable **M**, and join it to **Takes** so you keep only courses those students take. Then return the course **title** (you will also need **Courses**).

(c) A classmate writes the inner query in the **FROM** clause but *forgets the tuple-variable*. In one sentence, why does this fail?

.....

Box 2 Bag or Set? (4 min)

For each query, write its result *as a bag* (keep duplicates), then state how many rows it has.

```
SELECT major FROM Students;
```

```
SELECT DISTINCT major FROM Students;
```

One line: the two queries above gave different row counts. Which one treats the result as a *set*, and why?

.....

Box 3 Aggregation (6 min)

Write SQL against the Side A schema.

(a) The number of *distinct* courses appearing in **Takes**.

(b) For each **major**, the average **gpa** of its students. Return the **major** alongside the average.

(c) A classmate writes `WHERE AVG(gpa) > 3.5` to keep high-average majors. Why is this rejected? (You will see the correct tool, `HAVING`, in a later lecture.)

.....

Box 4 IN, ANY, ALL (8 min, discuss)

None of these appeared in lecture work them out from the Side A reference.

(a) Use **IN** with a sub-query: the names of all students who take **CSC108**.

(b) Use `>= ALL` with a sub-query: the name(s) of the student with the highest **gpa**. *Hint: their gpa is $\geq$ every gpa.*

(c) `gpa > ANY (SELECT gpa FROM Students)`, which students does this keep, and which (if any) does it exclude? Answer in one line.

.....

Box 5 Putting It Together (8 min, discuss)

(a) *Aggregation inside a sub-query.* Return the name(s) of every student whose `gpa` is below the overall average `gpa`. The average must be computed by a sub-query (an aggregate cannot live in `WHERE`).

(b) *Two tables.* Return the `title` of every course taken by a student named `Anya`. You will need `Students`, `Takes`, and `Courses` together.

(c) *Reflect.* Part (b) could also be written with a sub-query in the `FROM` clause, or with `IN`. In one or two lines, sketch *one* alternative, you need not write full SQL.

.....
.....